Undecidability

Maximilian Siemers

Version: 2, 2026-03-10

Course mechanics

Instructor:	Max(imilian Siemers)
E-Mail:	msiemers@cmu.edu
Meetings:	Tuesday/Thursday 11-12.20, Wean 5328
Lecture notes:	Link on Canvas. Will be updated frequently! (work in progress)
Office hours:	Thursday 3pm, Baker Hall 155E
Assignments:	(Bi-)Weekly, posted on Canvas. Assigned Thursdays, due Thursdays **before class**
Assessment:	Assignments: 35%, Final exam: 15%
Final exam time:	Friday, May 1st, 8.30-11.30am
Final exam mode:	Individual oral exam

Sources

I will make lecture notes available on Canvas. These are work in progress and will be updated frequently.

The lectures notes are based on the following sources, which I will reference using the indicated codes:

- [BBJ] Boolos, G. S., Burgess, J. P., and Jeffrey, R. (2007): *Computability and Logic* (5th ed.). Cambrige University Press
- [IC] Open Logic Project (2025): *Incompleteness and Computability*

  Available here: https://ic.openlogicproject.org/
- [J] Avigad, Jeremy (2007): *Computability and Incompleteness. Lecture notes*

  Available here: https://www.andrew.cmu.edu/user/avigad/Teaching/candi_notes.pdf

Contents

1 Introduction **3**

2 Effective computability **6**

3 Turing computability **7**

3.1 Turing machines 7

3.2 Monadic Turing machines 9

3.3 Computations of Turing machines 9

3.4 Turing's thesis 13

4 Enumeration and diagonalization **15**

5 The halting problem **15**

6 Recursion **15**

7 Turing computable and recursive functions are the same **15**

8 Universal computation **15**

9 Computable sets and reducibility **15**

10 Undecidability of the Entscheidungsproblem **15**

11 Church-Turing Thesis **15**

12 Potential further topics **15**

12.1 The smn theorem 15

12.2 Rice's theorem 15

12.3 Representability in arithmetic as a model of computation 15

12.4 Fixed point theorem 15

12.5 Kleene's recursion theorem 15

12.6 Incompleteness via halting problem 15

A toy example

Consider the following little puzzle from Douglas Hofstadter's *Gödel, Escher, Bach* (1979): There are three symbols M, I, and U and four rules:

1. $xI \to xIU$

   (Add a U to the end of any string ending in I. Example: MI to MIU)

2. $Mx \to Mxx$

   (Double the string after an initial M. Example: MIU to $MIUIU$)

3. $xIIIy \to xUy$

   (Replace any III with a U. Example: $MUIIIU$ to $MUUU$)

4. $xUUy \to xy$

   (Remove any UU. Example: $MUUU$ to MU)

Question: Using these rules, can you transform the string MI to the string MU?

1 Introduction

In the first half of this course, you proved Gödel's Incompleteness Theorems, two groundbreaking limitational results that concern certain formal systems. These results say something about what can *not* be proved in formal systems that contain basic arithmetic (Robinson's Q including zero, successor, addition, multiplication). You saw that basic arithmetic is expressive enough to "talk about itself" in a certain sense: for example, the notion of "proof", "provability", and "consistency" can be *represented* by formulas in the language of arithmetic. This gives rise to certain paradoxa essentially due to self-reference. For example, it is possible to formulate *in the language of arithmetic* so-called *Gödel sentences*, which essentially say: "This sentence is not provable". And so we get:

Theorem 1.1 (Gödel's First Incompleteness Theorem)**.** *There is no consistent, complete, axiomatizable extension of Q.*

Theorem 1.2 (Gödel's Second Incompleteness Theorem)**.** *No consistent axiomatizable extension of Q derives its own consistency sentence.*

These results state limitations of what can be *proved* in logical systems rich enough to talk about arithmetic. In the second half of the course, we shift focus and investigate another aspect of logical systems: namely, to what extent we can devise algorithms that can *compute* certain properties of these systems or answer certain questions about them.

The central result we will prove is the *unsolvability of the decision problem for first-order logic*. The decision problem for first-order logic in its modern form was posed by German mathematician **David Hilbert** in 1928, but it traces back farther in history. Already in the 17th century, German philosopher and polymath **Gottfried Leibniz** dreamt of a machine that can automatically and through mere symbol manipulation decide for any given mathematical statement whether it was true or false. In the 1920s and 1930s, the decision problem was one of the most important open questions in the foundations of math, and occupied the minds of the leading mathematicians of the time – in particular those around David Hilbert in Göttingen. Since that time, it is – even in English – often called by its German name: **Entscheidungsproblem**.

> **Entscheidungsproblem (Decision problem for first-order logic)**.
> Is there an algorithm that, given any sentence φ in the language of first-order logic, determines whether φ valid – i.e., true in all models?

Recall the **soundness** and **completeness** results for first-order logic: A proof system for first-order logic (e.g., natural deduction) is **sound** if $\vdash \varphi \implies \vDash \varphi$ (if φ can be proved, then it is valid); it is **complete** if $\vDash \varphi \implies \vdash \varphi$ (if φ is valid, then it can be proved). Soundness and completeness together mean that $\vdash \varphi \iff \vDash \varphi$ – i.e., provability and validity coincide.

Thus, by soundness and completeness, the decision problem for first-order logic can be equivalently stated as follows:

> **Entscheidungsproblem (Decision problem for first-order logic)**.
> Is there an algorithm that, given any sentence φ in the language of first-order logic, determines whether φ is provable?

Think for a moment what it would mean if the answer to the Entscheidungsproblem was *yes*. Suppose there was a machine M that, given any first-order sentene φ as input, is guaranteed to correctly determine after a finite amount of time whether or not φ is provable (valid) – say, by outputting 1 if the answer is "yes", and 0 if the answer is "no". A moment of thought shows that, although there are infinitely many sentences in the language of first-order logic, we can *enumerate* them. That is, we can devise a list $\varphi_1, \varphi_2, \ldots$ that contains all sentences of first-order logic[1]. Given the machine M and such an enumeration of sentences, we can start with φ_1, "feed" it to M, and wait for its output - by assumption, M after a finite amount of time will output either 1 (if $\vdash \varphi$) or 0 (if $\nvdash \varphi$). Once that is done, we continue with φ_2, ad infinitum.

[1] This depends on the specific *language* or *signature* we're working with, and such an enumeration of all sentences of first-order logic is possibly only under (very reasonable) assumptions about the underlying language: namely, that the language contains at most a countable infinity of non-logical symbols (function symbols, predicate symbols, and constants).

Of course, this process will take an infinite amount of time, but we will reach each formula after a *finite* amount of time and obtain a decision whether or not that formula is provable/valid. So we would have a theorem-prover that would (given infinite time) eventually list all the provable sentences of first-order logic. It is easy to show that this extends to all finitely axiomatizable theories of first-order logic.

In the mid-1930s, Alonzo Church and Alan Turing independently proved that the **Entscheidungsproblem is unsolvable**, or, as one says, that **first-order logic is undecidable**: There is **no algorithm** that can decide for an arbitrary first-order sentence φ whether or not φ is valid (provable). This, together with Gödel's Incompleteness Theorems that came about five years earlier, was a **huge** blow to many of the leading mathematicians of the time, for it means that in a certain sense logic (and, by extension, mathematics) eludes "automatic" treatment through algorithms. And this result is what we're going to prove in the second half of this course!

Note that in my above formulation of the unsolvability of the Entscheidungsproblem, I said that there is *no algorithm* that can (...). I have not made clear what I mean by "algorithm", but the result depends on that notion. For the negative result of the unsolvability of the Entscheidungsproblem says that something can *not* be computed. To make sense of the claim that something is or is not computable, we need to know what it means to be computable. Indeed, in Turing's 1936 paper *On computable numbers, with an application to the Entscheidungsproblem*, he introduces *Turing machines*, which are now considered *the* universal model of computation, in the following sense: **If** something can be computed at all, then it can be computed by a Turing machine. (In fact, Turing machines even provided the theoretical foundations for modern-day all-purpose computers, the first instantiations of which were constructed a little later.)

En route to proving undecidability of first-order logic, we will prove the undecidability of the famous *Halting problem* for Turing machines, and be concerned with the more general question: *What is computation?* We will introduce Turing machines and prove undecidability using them, but we will also see various other models of computation and prove their equivalence. The equivalence of a large number of *very* different models of computation gives rise to the **Church-Turing Thesis**, which is a thesis about what, precisely, computation *is*.

Thus, we will also deal with the fundamental issue of the *nature of computation*, and see multiple other exapmles of *(computably) unsolvable problems*. Given that you are at CMU, you have probably heard of the Church-Turing Thesis before, and many of you should already be convinced by the relevance of this issue. But just in case, here are a few further motivations:

- Modern *complexity theory*, which, for example, provides the theoretical groundworks for many approaches in *crpytography*, in a certain sense extends *computability theory*: Whereas computability deals with questions of what, in principle, is or is not computable, complexity theory adds constraints concerning

time and memory use in computation. The methods of complexity theory basically extend those of computability theory. Any of you who have taken a course in complexity theory will probably have dealt a lot with proofs involving Turing machines.

- Limits to *automated theorem proving*: Undecidability of first-order logic says that there can be no automated procedure that decides whether a given sentence of first-order logic is provable. Thus, there can also be no fully automated procedure that decides whether a given mathematical statement (formalized in first-order logic) is or is not a theorem.

- Limits to *software verification*: Undecidability of first-order logic has direct limitational consequences for the possibility to decise automated procedures that answer questions about computer programs such as:

  - Will this program ever crash?
  - Will this program ever reach a bad state?
  - Are these two programs equivalent?

  If time allows, we will prove *Rice's Theorem*, which states roughly that no non-trivial semantic property of programs is decidable.

- Scope of Gödel's incompleteness theorems: As stated above, Gödel's incompleteness theorems state something about *axiomatizable* theories. A theory is *axiomatizable* if there exists an algorithm that, for a given sentence, decides after a finite amount of time whether or not is is an axiom of the theory. Thus, the scope of Gödel's theorems directly depends on the question: *What is computation?*

One last remark before we really get started: Remember that, in proving incompleteness, we used the fact that basic arithmetic is expressive enough to express meta-logical properties of theories, derivations, provability, etc. and thus about itself. Something conceptually very simular is going on in the proof of the undecidability of the Halting problem and the undecidability of first-order logic: namely, first-order logic is expressive enough to "talk about" computation, as we will make precise.

2 Effective computability

Since the central results we will be concerned with in this part of the course are about problems that *can* or can *not* be solved by algorithms, we should first state what we mean by an *algorithm*. Intuitively, an algorithm (or a computational procedure) consists of a *recipe*, or a set of *instructions* that one (a person *or* a machine) can follow to obtain an answer to a question, or an output given an input.

So we are looking for a characterization of certain ("computable") maps from questions to answers, or from inputs to outputs. Thus it makes sense to think of algorithms as certain kinds of functions, and the question becomes: *what properties must a function have to be called "computable"*? For simplicity, let's just think about functions from natural numbers to natural numbers[2].

Historically, functions which are intuitively computable were called "effectively computable" or just "effective" functions. Think of this as meaning the same as "intuitively computable" functions. So – what is a "computable" function? Here is a nice description from [BBJ, p. 23]:

> A function f from positive integers to positive integers is called *effectively computable* if a list of instructions can be given that in principle make it possible to determine the value $f(n)$ for any argument n. (This notion extends in an obvious way to two-place and many-place functions.) The instructions must be completely definite and explicit. They should tell you at each step
>
> - what to do,
> - not tell you to go ask someone else what to do,
> - or to figure out for yourself what to do:
>
> the instructions should require no external sources of information, and should require no ingenuity to execute, so that one might hope to automate the process of applying the rules, and have it performed by some mechanical device.

In the next section, we will introduce one notion to make precise this intuitive definition of what it means to be "computable": the *Turing machine*. It will be clear that any Function computable by a Turing machine is also "computable" in the intuitive sense. The converse of this claim – namely, that any intuitively "computable" function can be computed by a Turing machine – is **Turing's Thesis**. We will get back to **Turing's Thesis** (and the **Church-Turing Thesis**) later in the course.

3 Turing computability

3.1 Turing machines

I will explain Turing machines and draw a picture in class. Here is the formal definition:

[2] As we will see later in the course, in the context of "computability" it is completely sufficient to restrict attention to functions of natural numbers. That is because basic arithmetic is, in a certain sense, expressive enough to encapsulate computation.

Definition 3.1 (Turing machine)**.** A *Turing machine (TM)* is a quadruple $T = (Q, \Gamma, \delta, q_0)$, where

- Q is a finite set of *states*;
- Γ is a finite set of *symbols* including a symbol B called "blank";
- $\delta : Q \times \Gamma \rightharpoonup Q \times \Gamma \cup \{L, R\}$ is a partial function called the *transitioning function*; and
- $q_0 \in Q$ is a state called the *initial state*.

A Turing machine operates on a one-dimensional *tape* which is devided into *cells*. On each cell exactly one symbol from Γ is written. The tape extends infinitely in both directions. It doesn't in fact matter whether we think of the tape as in fact *infinite* or imagine that it is finite, but a little man is standing at each end and, whenever the Turing machine tries to move beyond the current end of the tape, glues more cells to the tape end as needed. Only a finite number of cells on the tape contain a symbol other than B – i.e., it is "blank" (or "empty") everywhere except in finitely many places.

We can think of a Turing machine $T = (Q, \Gamma, \delta, q_0)$ operating on the tape as follows: T has a "reading head" which, at each point in time, is positioned on top of one cell of the tape. T is started in its initial state q_0 on some cell. At each time point, it is in some internal state $q \in Q$ and can "read" the symbol written on the cell under its reading head. The action performed by T in one step is specified by the transitioning function δ and is thus determined by the current internal state q and the symbol under its head. At each step, T can perform one of the following actions:

1. If the current state is q and the symbol read is γ, then set the new state to q' and replace γ by γ'.
2. If the current state is q and the symbol read is γ, then set the new state to q' and move to the *L*eft.
3. If the current state is q and the symbol read is γ, then set the new state to q' and move to the *R*ight.

Note that δ is a partial function. This means that for each combination (q, γ) of a state q and a symbol γ there is *at most* one instruction that applies to T in any situation. Thus, a fourth action of T (better described as a non-action) is: do nothing, if there is no instruction that applies. We say that T *halts* when this happens.

3.1.1 Specifying Turing machines

For a given TM $T = (Q, \Gamma, \delta, q_0)$, each of the three actions described in the previous section corresponds to an assignment $(q, \gamma) \mapsto (q', a)$ of δ, where $a \in \Gamma \cup \{L, R\}$. Note that by definition of a TM, the set of states Q and the tape alphabet Γ is finite, so the instructions given by δ can be represented as a finite lookup table or as a flowchart.

In the table representation, columns correspond to states Q, and rows correspond to symbols from the alphabet Γ. Each cell of the table contains two symbols: One state, and either one symbol from Γ or an element of $\{L, R\}$.

In the diagram representation, there are circles representing states, and arrows between states representing actions. The arrows are labelled with two symbols separated by a colon (:).

The meaning of the symbols inside the table cells and in the labels of the diagram arrows is as follows: The left symbol is the symbol on the tape that the TM needs to read to trigger this action. The right symbol is either an element of Γ, if the instruction is to write a symbol, or an element of $\{L, R\}$, if the instruction is to move *L*eft or *R*ight.

The lack of an action for a combination (q, γ) of a state and a symbol corresponds to halting and is represented in tables as an empty cell, and in diagrams as lack of a corresponding arrow.

3.2 Monadic Turing machines

So far, we have defined a Turing machine $T = (Q, \Gamma, \delta, q_0)$, where Γ is any finite "alphabet", or set of symbols that can be written in the cells of the tape that the Turing machine operates on, as long as Γ contains the "blank" symbol B.

For the time being, we will simplify this definition, and only consider *monadic* Turing machines, where $\Gamma = \{1, B\}$. That is, for such a Turing machine, each cell of the tape contains either a stroke (1) or is blank (B). The following discussion is based on monadic Turing machines of this kind.

3.3 Computations of Turing machines

Our goal for this section is to define what computations of Turing machines are, and what it means for a Turing machine to compute a function on the natural numbers.

3.3.1 Configurations

We saw that the next *step* of a Turing machine T is determined by its current state q and the symbol under the tape head γ. The *entire future behavior* of T is determined by:

- the current internal state q;

- the tape content; and
- the position of the reading head on the tape.

Since we assumed that T is started on a tape which contains only a finite number of non-blank symbols, the tape content at each step can be described as a finite string of symbols (from the first to the last non-blank symbol on the tape). Thus, we can represent a *configuration* of T as follows.

Definition 3.2 (Configuration)**.** A *configuration* of a Turing machine $T = (Q, \Gamma, \delta, q_0)$ is a quadruple $(\alpha, q, \gamma, \beta)$, where

- α is the string of symbols to the *left* of the reading head of T starting from the first non-blank symbol of the tape if there *are* non-blank symbols to the left of the reading head or B otherwise;
- β is the string of symbols to the *right* of the reading head of T ending at the last non-blank symbol of the tape if there *are* non-blank symbols to to the right of the reading head or B otherwise;
- $\gamma \in \Gamma$ is the symbol *under* the reading head of T; and
- $q \in Q$ is the current *state* of T.

A convenient way of writing down configurations is to write out the string $\alpha\gamma\beta$ and attach the state q as a subscript to γ. For example, 1101_q0011 denotes the configuration $(110, q, 1, 0011)$.

The next definition tells us when a Turing machine stops operating.

Definition 3.3 (Halting configuration)**.** A configuration $C = (\alpha, q, \gamma, \beta)$ is a *halting configuration* of a TM $T = (Q, \Gamma, \delta, q_0)$ if no instruction of δ applies to C – i.e., if $\delta(q, \gamma)$ is undefined.

For any configuration that is not a halting configuration, when can determine what configuration comes next by checking the instructions in δ.

Definition 3.4 (Next configuration)**.** Given a TM $T = (Q, \Gamma, \delta, q_0)$ and a non-halting configuration $C = (\alpha, q, \gamma, \beta)$, the *next configuration* C' or the *configuration* C' *after* C is determined as follows:

- "Write instruction": If $\delta(q, \gamma) = (q', \gamma')$ for some $q' \in Q$ and $\gamma' \in \Gamma$, then $C' = (\alpha, q', \gamma', \beta)$.
- "Move left instruction": If $\delta(q, \gamma) = (q', L)$ for some $q' \in Q$, then $C' = (\alpha', q', \gamma\beta)$, where α' is all of α except its *last* symbol if the length of α is at least 2, and B otherwise.
- "Move right instruction": If $\delta(q, \gamma) = (q', R)$ for some $q' \in Q$, then $C' = (\alpha\gamma, q', \gamma')$, where γ' is all of γ except its *first* symbol if the length of γ is at least 2, and B otherwise.

3.3.2 Representing inputs and outputs

Almost there! Before we can define what it means for a Turing machine to compute a function, we need some conventions about how we represent inputs and outputs.

The idea is that we represent a natural number n as a block of n strokes (1s) on the tape. However, this makes it somewhat confusing what the representation of 0 is – so it is a better idea to use a block of $n+1$ strokes to represent n.

If we want to pass more than one input to a TM, we simply separate the blocks corresponding to the inputs by a single blank (B). Thus, the input of the three arguments $3, 0, 2$ is represented by

$$1111B1B111.$$

We represent outputs the same way.

We adopt the convention that, for an input $(x_1, \ldots, x_n)$ of n natural numbers, we represent $(x_1, \ldots, x_n)$ on the tape as described above, and otherwise the tape is empty (blank). In the beginning of its computation, the Turing machine scans the leftmost stroke of the block for x_1, and its state is q_0.

Similarly, we adopt the convention that a configuration of T represents an ouput $y \in \mathbb{N}$ if the tape contains a block of $y+1$ strokes and is otherwise blank, T is scanning the leftmost stroke of this block, and T is halted – i.e., it is in a state q such that no instruction of δ applies.

The next two definitions capture these conventions formally. For a natural number n, write 1^n for a block of n 1s. For example, $1^3 = 111$.

Definition 3.5 (Initial configuration for an input)**.** Given a monadic TM T and natural numbers $x_1, \ldots, x_n$, the *initial configuration of T for input $x_1, \ldots, x_n$* is the configuration

$$1_{q_0} 1^{x_1} B 1^{x_2+1} B \cdots 1^{x_n+1}.$$

Definition 3.6 (Final configuration for an output)**.** Given a monadic TM T and a natural number y, a *final configuration of T for output y* is a halting configuration

$$1_q 1^y.$$

(Recall 3.3: The configuration $1_q 1^y$ is a halting configuration if $\delta(q, 1)$ is undefined.)

3.3.3 Function computed by a Turing machine

Now we're *really* almost there.

Definition 3.7 (Computation sequence, halting)**.** Let T be a Turing machine and $(x_1, \ldots, x_n)$ an n-tuple of natural numbers. A *partial computation sequence of T for input $(x_1, \ldots, x_n)$* is a sequence of configurations $C_1, C_2, \ldots, C_k$ such that:

- C_0 is the initial configuration for input $(x_1, \ldots, x_n)$;
- for each $i < k$, C_{i+1} is the configuration after C_i (according to T, see Definition 3.4).

A *halting* computation sequence of T for input $(x_1, \ldots, x_n)$ is a partial computation sequence where the last configuration is a halting configuration.

We say that T *halts on input* $(x_1, \ldots, x_n)$ iff there exists a halting computation of T for input $(x_1, \ldots, x_n)$.

Finally!

Definition 3.8 (Parial function computed by a TM)**.** Let T be a monadic TM and f an n-ary *partial* function from $\mathbb{N}$ to $\mathbb{N}$. We say that T *computes the partial function* f if the following holds:

For every n-tuple of natural numbers $(x_1, \ldots, x_n)$, if T is started in the initial configuration for $(x_1, \ldots, x_n)$, then

- if $f(x_1, \ldots, x_n)$ is defined and $f(x_1, \ldots, x_n) = y$, then T halts in a final configuration for y;
- if $f(x_1, \ldots, x_n)$ is undefined, then T either does not halt or it halts in a non-final configuration for y.

So a TM T computing a partial function f started on the representation of an input for which f is undefined either:

- does not halt, i.e. it "loops" forever; or
- it does halt, but then it halts in a non-final configuration. According to Definition 3.6, this is the case when one of the following happens:
  - T halts on an entirely empty tape;
  - T halts on a tape that contains more than one consecutive block of 1s; or
  - T

The following definition is just a special case of Definition 3.8.

Definition 3.9 (Total function computed by a TM)**.** Let T be a monadic TM and f an n-ary *total* function from $\mathbb{N}$ to $\mathbb{N}$. We say that T *computes the total function* f if:

For every n-tuple of natural numbers $(x_1, \ldots, x_n)$, if T is started in the initial configuration for $(x_1, \ldots, x_n)$, then T halts in a final configuration for $f(x_1, \ldots, x_n)$.

Definition 3.10. An n-ary total or partial function from $\mathbb{N}$ to $\mathbb{N}$ is *Turing-computable* if there exists a Turing machine that computes it.

Theorem 3.11. *The following functions are Turing computable:*

1. $f(x) = x + 1$
2. $g(x, y) = x + y$
3. $h(x, y) = x \times y$

Proof. Exercise. We will do one of them in class. □

3.4 Turing's thesis

If you have seen definitions of Turing machines before, they might have been different from the one I gave above. Here are some common variations:

- Allow an arbitrary finite alphabet Γ instead of just the two symbols 1 and B.
- TMs with a total instead of a partial transitioning function δ with a designated set of *halting states*.
- Tape infinite in only one direction.
- TMs that can read and write simultaneously on more than one tape.
- TMs that operate on a two-dimensional grid rather than a one-dimensional tape.
- Nondeterministic TMs that allow for more than one applicable instruction at each step. Here, the TM can nondeterministically "choose" what to do next.

It turns out that all these different ways of defining a Turing machine are invariant with respect to the class of Turing computable functions – that is, one can prove that the examples I gave above all compute *the same functions*.

(**Mini-excursion into complexity theory.** There *are* complexity-related differences between different ways of defining Turing machines: The above variations of Turing machines differ in how much time or memory they need to compute certain functions. In fact, the famous P-vs-NP problem of theoretical computer science essentially boils down to different definitions of a Turing machine:

P is defined as the class of decision problems that can be solved by a deterministic Turing machine (just like the one given in Definition 3.1) in polynomial time relative to input size. NP is defined as the class of decision problems that can be solved by a *non*-deterministic Turing machine in polynomial time relative to input size. It is an open question whether or not $\mathsf{P} = \mathsf{NP}$. Thankfully, in computability theory, we do not need to care about time or space complexity, so for us deterministic and nondeterministic Turing machines are equivalent.)

It is not too hard to prove that all the above variations of Turing machine compute the same functions.

It should also be clear that every Turing-computable function is *effectively computable* in the sense of Section 2: What a TM does at each step is determined by a finite list of instructions. I should be able (on a good day) to simulate the actions of a Turing machine, and it doesn't require much coding competence to write a program in, for example, Python, that does the same.

The converse claim that *any effectively computable function* as discussed in Section 2 is Turing computable called *Turing's Thesis*. Note that this is a bold claim: While we have seen that addition and multiplication of natural numbers are Turing-computable, Turing's Thesis implies that for *any* function that we intuitively deem computable at all there exists a Turing machine that computes it.

Turing's thesis cannot be directly proved, because it is a claim that involves the intuitive, unformalized notion of *effective* (or *intuitive*) computability. There are, however, arguments that can be given for it. We will return to this issue in the section on the Church-Turing thesis.

Problems

1. Prove that $f(x) = x + 1$ is Turing-computable.
2. Prove that $g(x, y) = x + y$ is Turing-computable.
3. Prove that $h(x, y) = x \times y$ is Turing-computable.
4. Prove that $i(x, y) = \min(x, y)$ is Turing-computable.
5. Prove that $j(x, y) = \max(x, y)$ is Turing-computable.

4 Enumeration and diagonalization

5 The halting problem

6 Recursion

7 Turing computable and recursive functions are the same

8 Universal computation

9 Computable sets and reducibility

10 Undecidability of the Entscheidungsproblem

11 Church-Turing Thesis

12 Potential further topics

12.1 The smn theorem

12.2 Rice's theorem

12.3 Representability in arithmetic as a model of computation

12.4 Fixed point theorem

12.5 Kleene's recursion theorem

12.6 Incompleteness via halting problem